

Lab: 0001

Verilog Setup & Basic Gate Implementation.

Where It All Begins

*The journey from a single transistor to the GPUs that train modern AI
— starting with 1s and 0s.*

CSA222: Modern Computer Architecture

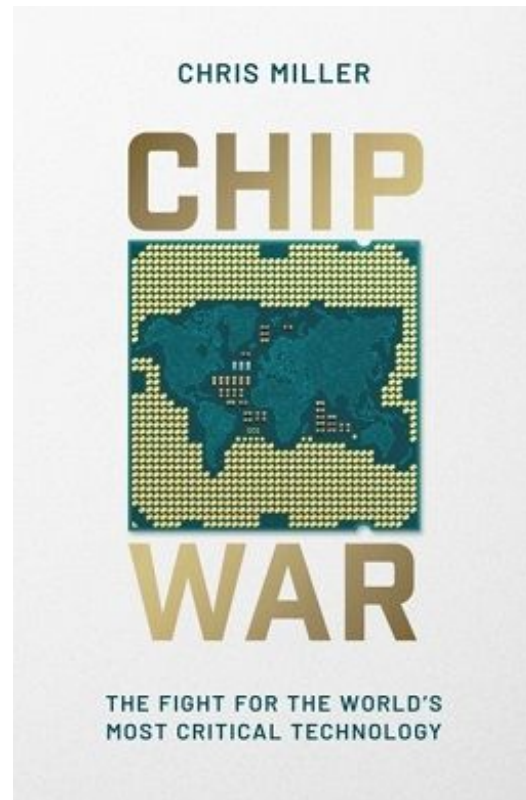
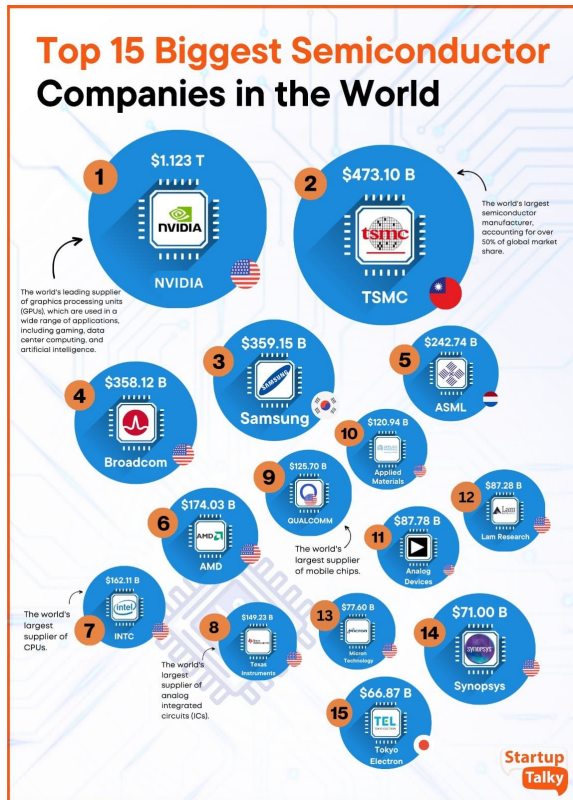
Why Hardware Description Languages Exist?

The mental-model shift everything else in this lecture depends on.

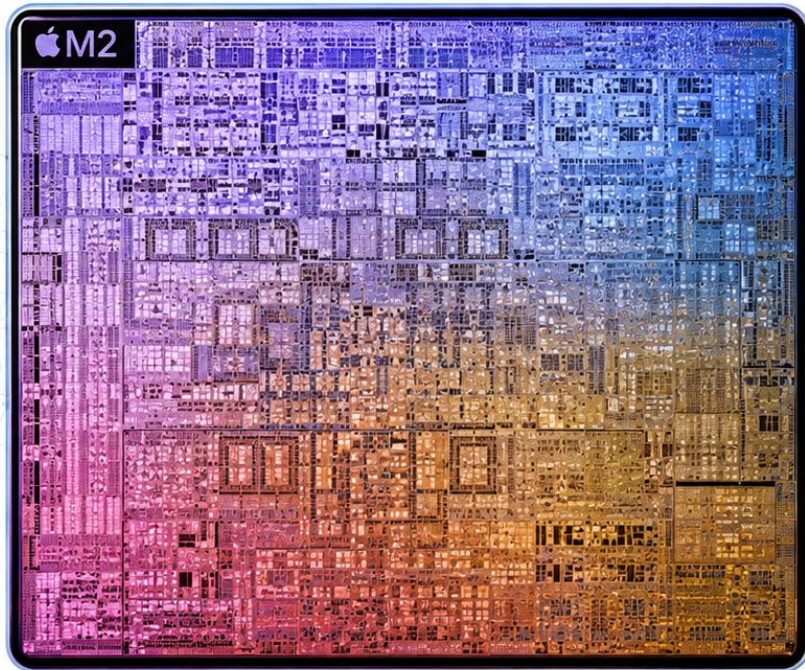
ARNOLD THACKRAY, DAVID C. BROCK,
and RACHEL JONES

Moore's Law

The Life of Gordon Moore,
Silicon Valley's Quiet Revolutionary

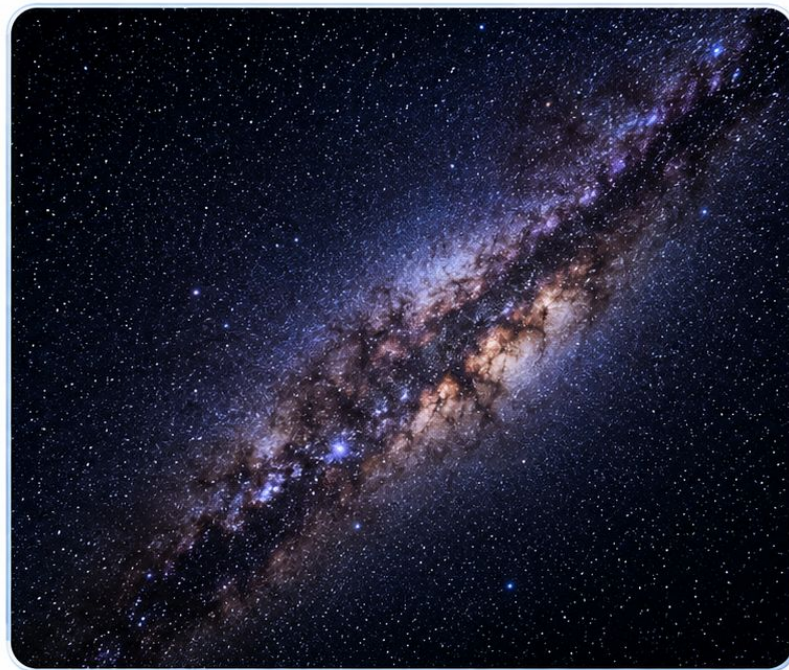


20 Billion Switches, One Idea



~20 billion transistors

Inside a modern Apple M2 chip.

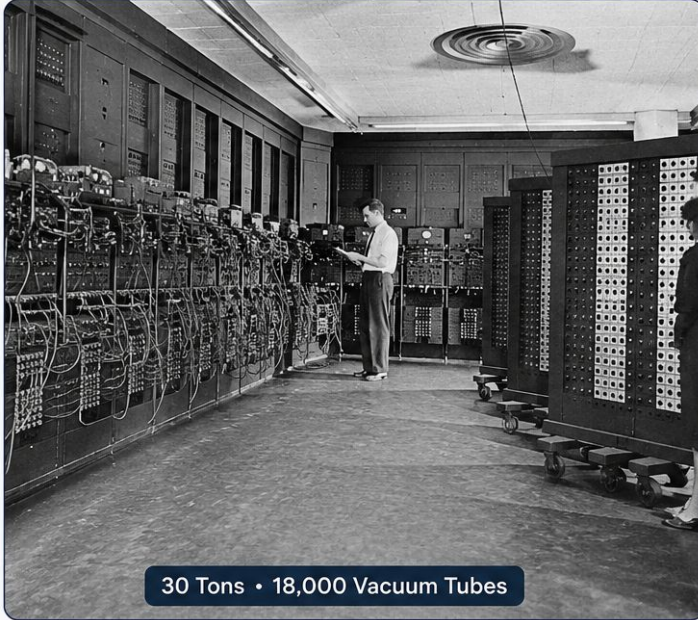


~100–400 billion stars

in our galaxy (the Milky Way).

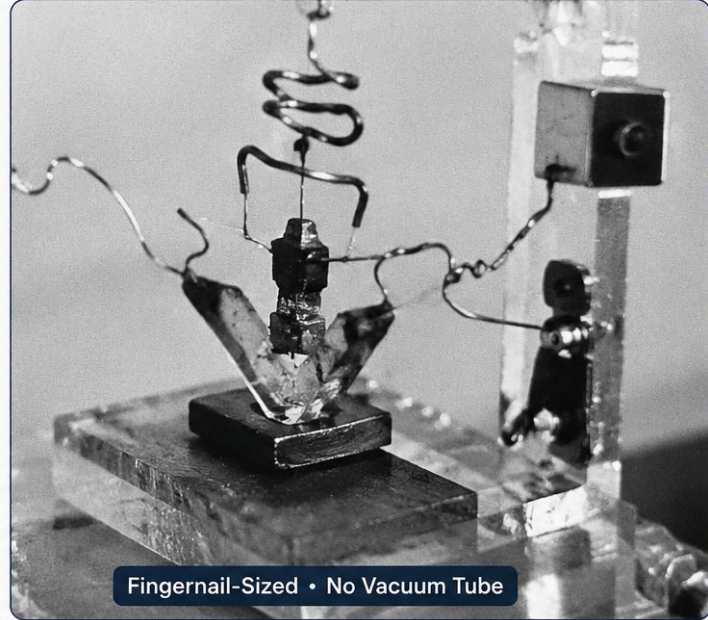
1947: The Switch That Replaced the Vacuum Tube

Before: ENIAC (1945)



30 Tons • 18,000 Vacuum Tubes

After: First Point-Contact Transistor (1947)

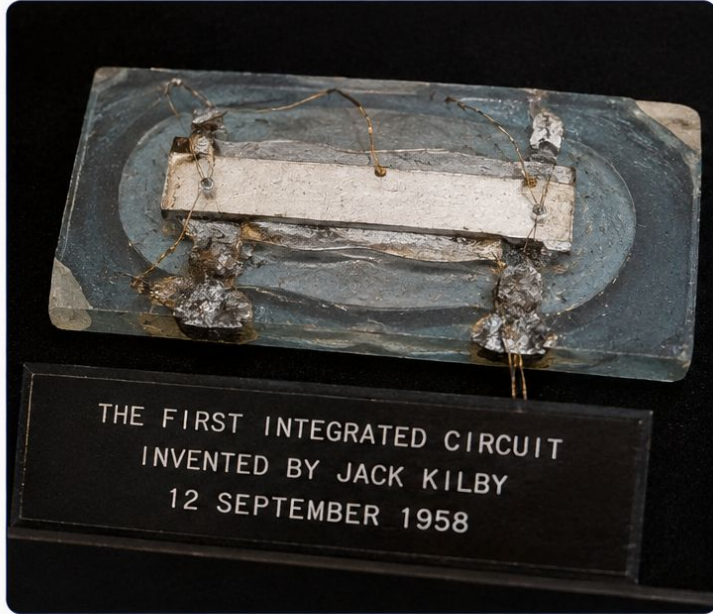


Fingernail-Sized • No Vacuum Tube

Room-sized → fingernail-sized, in one invention.

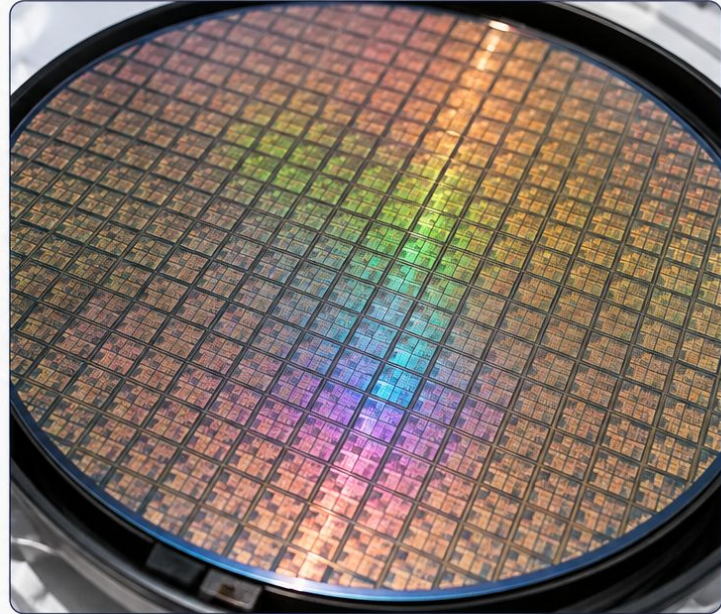
The Integrated Circuit: From Wires to Wafers

1958 – Kilby's Original Integrated Circuit



First IC: 1 transistor.

Modern Silicon Wafer (nm-scale chips)

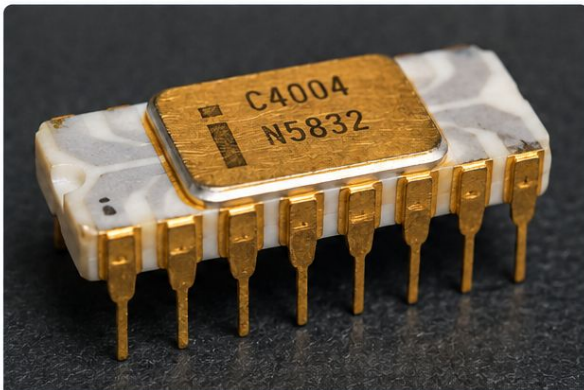


A modern wafer: trillions.

1971: The First CPU on a Single Chip

Intel 4004 Microprocessor (1971)

The world's first commercially available microprocessor.



Transistor Count: ~2,300



Clock Speed: ~740 kHz



Word Length: 4-bit



Technology: 10 μ m PMOS

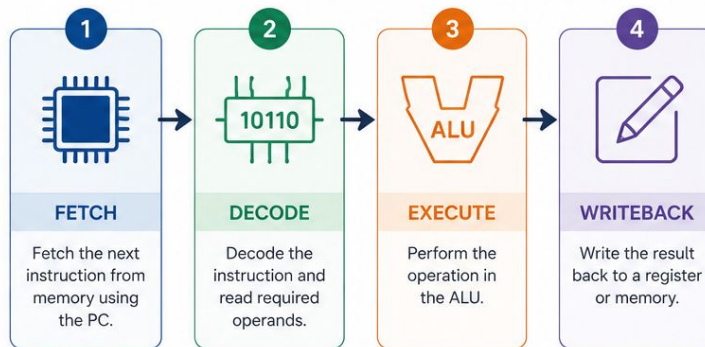


Introduced: November 1971

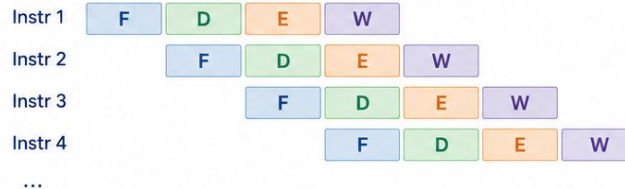
A complete CPU on a chip — the beginning of a revolution.

Foreshadowing: The 4-Stage Pipeline

The foundation of modern CPU performance.

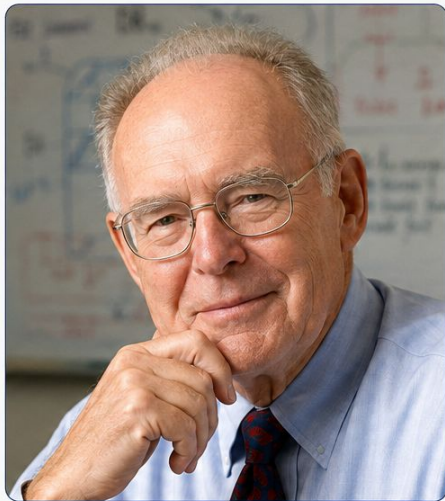


Time



While one instruction is executing, others are being fetched, decoded, and written back — increasing throughput.

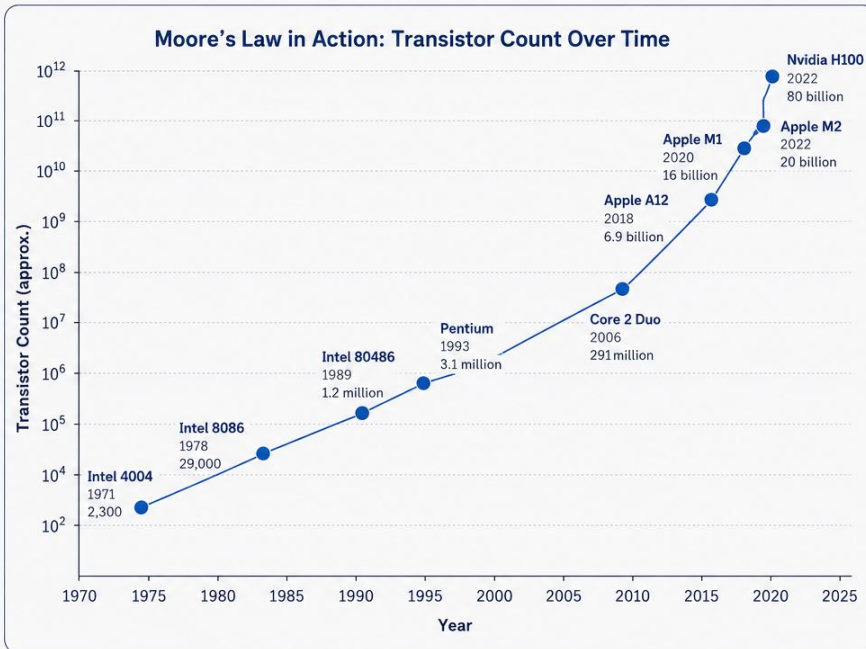
Moore's Law: The Prediction That Ran the Industry



Gordon Moore

Intel Co-founder

“ The number of transistors incorporated in a chip will approximately double every two years. ”
– 1965

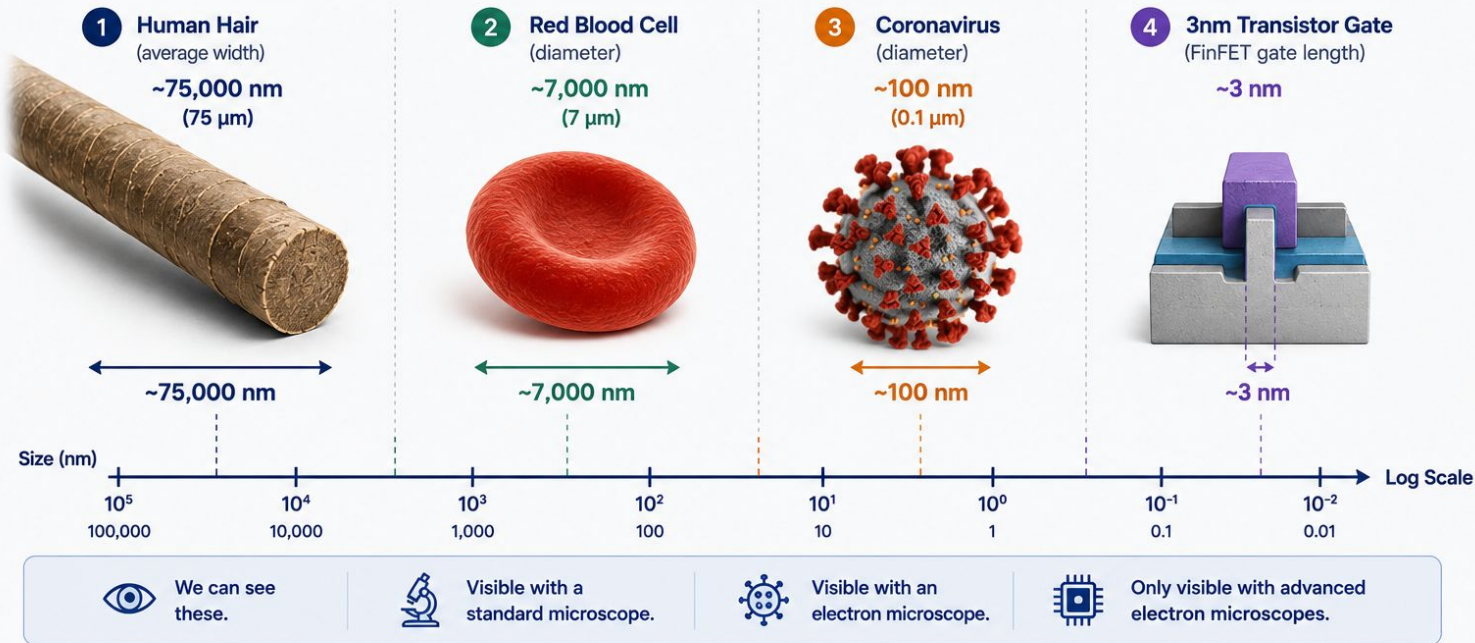


Chip	Intel 4004	Intel 8086	Intel 80486	Pentium	Core 2 Duo	Apple A12	Apple M1	Apple M2	Nvidia H100
Year	1971	1978	1989	1993	2006	2018	2020	2022	2022
Transistor Count (approx.)	2,300	29,000	1.2 million	3.1 million	291 million	6.9 billion	16 billion	20 billion	80 billion

How Small Is 3nm, Really?

How Small Is a Transistor? A Scale Comparison

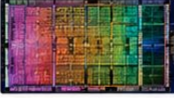
From things we can see, to things we can't.



From ~75,000 nm to ~3 nm — a size journey of over 4 orders of magnitude.
This is the scale that makes modern computing possible.

1 nm = 0.000000001 m
(one billionth of a meter)

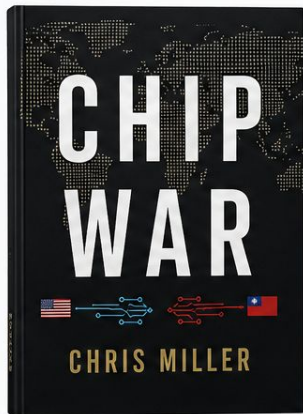
From A Few Gates to Billions

ERA	GATE COUNT (APPROX.)	REPRESENTATIVE CHIP
 1. Vacuum Tube Era 1940s – Early 1950s	$10^2 - 10^3$ (Hundreds to Thousands)	 ENIAC ~1,500 gates (1945)
 2. Transistor Era Mid 1950s – 1960s	$10^3 - 10^5$ (Thousands to Hundred Thousands)	 TI 7400 (TTL) ~4,000 gates (1968)
 3. Integrated Circuit Era 1970s – 1980s	$10^5 - 10^6$ (Hundred Thousands to Millions)	 Intel 8086 ~29,000 gates (1978)
 4. VLSI Era 1990s – 2000s	$10^6 - 10^9$ (Millions to Billions)	 Intel Core i7 (1st Gen) ~1.17 Billion gates (2008)
 5. Modern VLSI Era 2010s – Today	$10^9 - 10^{11}+$ (Billions to Hundreds of Billions)	 Apple M2 ~20 Billion gates (2022)



The number of gates on a chip has grown from a few hundred to billions — enabling the **computers, smartphones, and AI systems we use today.**

Why One Island Makes Almost Every Advanced Chip



Big Idea

Advanced chips sit at the center of economic power and national security.



The countries that design, make, and control chips shape the future.



Supply chains are global, but chokepoints are real.

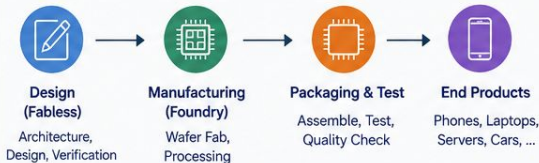


The next decades will be defined by chip innovation — and chip independence.

Where the World's Most Advanced Chips Are Made



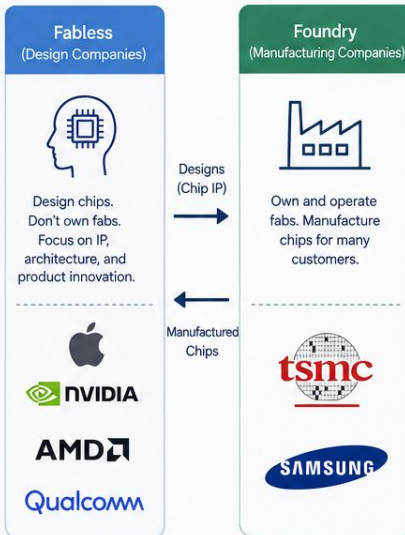
How a Chip Reaches Your Device



Design can happen anywhere. Manufacturing at the cutting edge happens in very few places.

Fabless vs. Foundry

Two sides of the same ecosystem



Chips power everything. Whoever leads in chips, leads the future.

Understanding this ecosystem helps us build the next generation of computing.

Apple Silicon: One Chip, Every Concept in This Course



The next generation of Apple silicon.

M2 integrates more than 20 billion transistors on a single chip, unifying high-performance CPU, stunning GPU, powerful Neural Engine, and more — with industry-leading power efficiency.



20 billion+ transistors



Second-generation 5 nm technology



High performance. Incredible efficiency.

GPU

Up to 10 cores

High-performance graphics for immersive experiences.

High-performance CPU cores
4 cores

High-efficiency CPU cores
4 cores

Unified memory controller
High-bandwidth, low-latency unified memory architecture for CPU, GPU, and Neural Engine.

Neural Engine
16-core

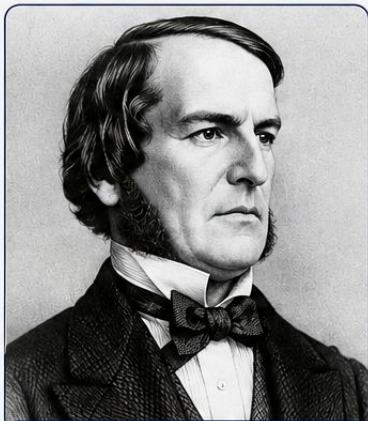
Machine learning accelerator for on-device AI.

System cache

Large cache for low-latency performance.

Die shot image captured by Apple.
© 2022 Apple Inc.

1847 + 1937: The Math That Was Waiting for the Transistor



GEORGE BOOLE

1815–1864

English mathematician and logician.

“The laws of thought are the same as the laws of algebra.”



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

**BOOLEAN
ALGEBRA**
(1847)

Boole formalized logic with variables that can be 0 or 1.



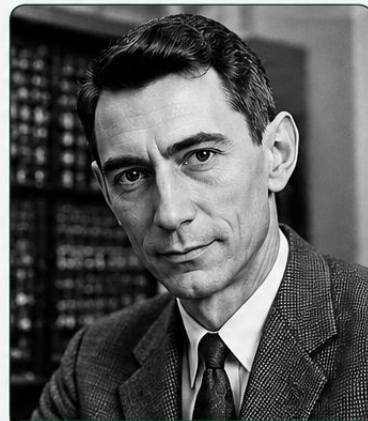
**SWITCHING
CIRCUIT THEORY**
(1937)

Shannon showed how logic could be built with switches and relays.



YOUR FIRST
assign y = a & b;
IN LAB 1.

Today, we write HDL. The math makes it predictable and powerful.



CLAUDE SHANNON

1916–2001

American mathematician, engineer, and information theorist.

“A digital circuit is a switching circuit.”



From ideas in the 19th century to circuits in the 20th, the foundation that powers every digital system you build.

Math → Theory → Silicon → Code



Boole gave us the language of logic.

Shannon showed us how to build with it.

You will use it to build amazing things.

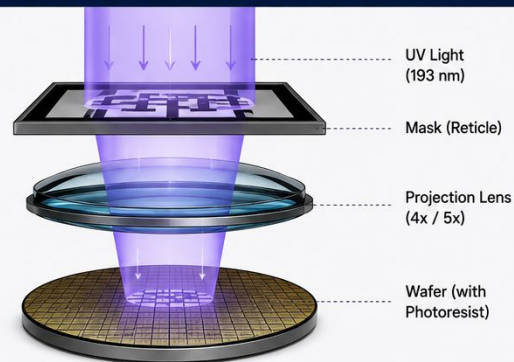
How a chip is actually manufactured?

1. SILICON INGOT & WAFERS

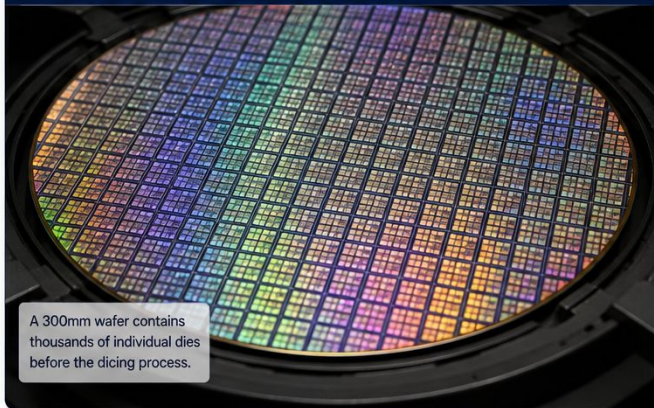


A single-crystal silicon ingot is grown using the Czochralski process and sliced into thin wafers (~775 μm thick).

2. PHOTOLITHOGRAPHY: LIGHT PATH



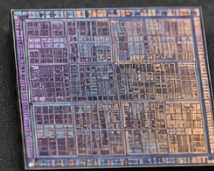
3. 300mm WAFER (BEFORE DICING)



A 300mm wafer contains thousands of individual dies before the dicing process.

4. BARE DIE vs PACKAGED CHIP

BARE DIE



The bare silicon die contains the active circuits.

PACKAGED CHIP

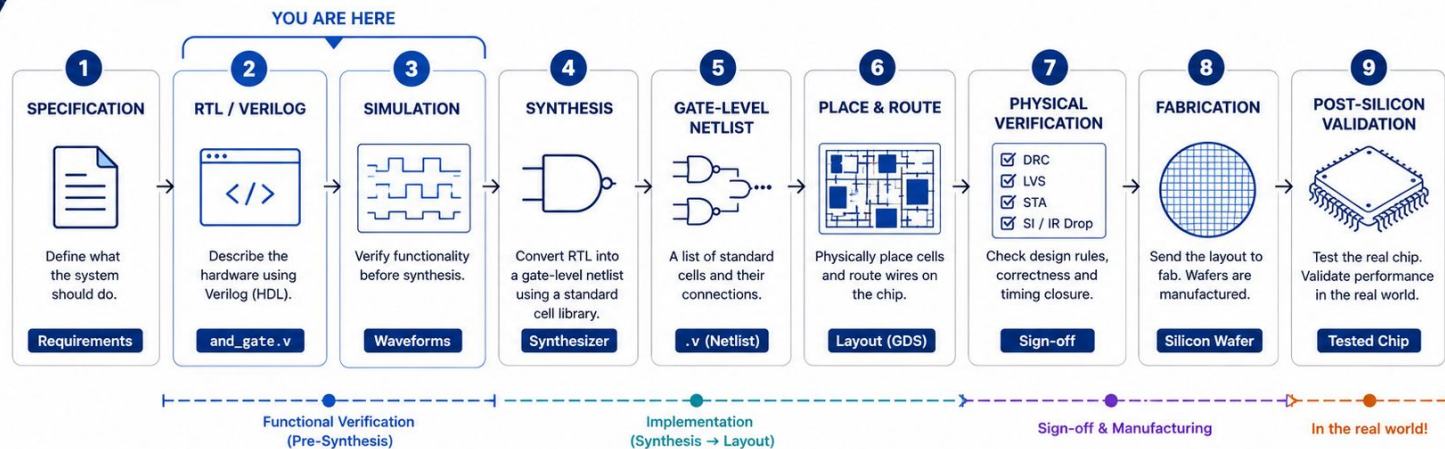


The die is mounted, connected, and protected inside a package for use in systems.

From Your First assign Statement to a Physical Chip: The Full Pipeline

From ``assign y = a & b;` to Silicon

The VLSI implementation flow — from your first line of Verilog to a real chip.



What you do in Lab 1

Write Verilog (RTL) for simple gates and simulate them to verify functionality.



and_gate.v



(Compile)



(Run)



Waveforms



Key Takeaway

HDLs let us describe hardware. Tools turn that description into working silicon.

“ Every billion-transistor chip you use started as a few lines of HDL.” — That’s the power of abstraction. ”

Prerequisites

This lecture assumes nothing about prior coding or HDL exposure.

L1

Boolean algebra recap

L1's four core gates and truth tables give the circuits we'll use as running examples — helpful, not required.

0

Zero coding background

No prior programming or HDL experience is assumed anywhere in this lecture. Every construct is introduced from first principles.

Everything here becomes muscle memory in Lab 1 — Verilog Setup & Basic Gate Implementation.

What You'll Build Today

By the end of this lab, you'll have all seven core gates running in simulation.

NOT

Inverts the input

AND

1 if both inputs are 1

OR

1 if either input is 1

NAND

0 if both inputs are 1

NOR

1 if both inputs are 0

XOR

1 if either input is 1

XNOR

1 if both inputs are same

PLUS

a single self-checking testbench, a generated VCD file, and a GTKWave screenshot of all four gates switching together.

Software Brain vs. Hardware Brain

One CPU executing instructions in order, versus thousands of gates switching at once.

Software: sequential

A program counter steps through instructions one at a time. "Turn on lamp 1, then lamp 2, then lamp 3" is a loop — each line waits for the last to finish.

```
for (i=0;i<4;i++) lamp[i]=1;
```

Hardware: concurrent

Four real lamps wired to four real switches all respond the instant power arrives — simultaneously, not in a queue. Verilog describes this concurrency directly.

```
assign lamp = switch; // x4, at once
```

Every bug this mismatch causes in Lab 1 traces back to reading Verilog like a to-do list instead of a wiring diagram.

Simulation vs. Synthesis

An HDL file is compiled two completely different ways, for two completely different purposes.



Simulation

Runs your Verilog on a normal CPU, modeling circuit behaviour over simulated time. Produces console output and waveforms. Nothing physical is built — this is everything Lab 1 does.



Synthesis

A synthesis tool (Synopsys, Cadence) reads the same style of Verilog and generates a real gate-level netlist — the actual circuit that gets fabricated on silicon.

Same language, two audiences: a simulator that pretends to be hardware, and a synthesizer that builds it for real.

Verilog, VHDL, SystemVerilog

Three HDLs exist in industry today. Here's why this course uses Verilog.

Verilog (1984)

C-like syntax, compact, the most widely taught HDL for digital design courses. IEEE 1364. What this course uses.

VHDL (1987)

Ada-derived, strongly typed, verbose. Dominant in defense, aerospace, and some European industry.

SystemVerilog (2005)

A superset of Verilog adding verification and object-oriented features. IEEE 1800 — where Verilog leads.

This course uses Verilog: compact syntax, the fastest on-ramp to writing your first working circuit.

A Brief History

Forty years old, still the industry's default teaching language for RTL design.

- **1984** Gateway Design Automation creates Verilog as a proprietary simulator language.
- **1989** Cadence acquires Gateway; Verilog spreads across the industry.
- **1995** IEEE 1364-1995 — Verilog becomes an open standard.
- **2001** IEEE 1364-2001 — ANSI-style ports, generate blocks, signed arithmetic.
- **2005** IEEE 1800 — SystemVerilog folds Verilog in as its foundation.

Verilog in the Real Flow

From the RTL you'll write to the silicon that ships.

RTL → Silicon

Every line of Verilog this lecture covers eventually passes through this exact pipeline.



Simulation (this whole lecture, and all of Lab 1) happens entirely at the RTL stage — before anything is synthesized, placed, or fabricated.



Tools of the trade

Simulators: Icarus Verilog, Synopsys VCS, Siemens Questa. Synthesis: Synopsys Design Compiler, Cadence Genus. Waveform viewers: GTKWave. Lab 1 uses the free, open-source column of this list.

Program Structure

The container every Verilog design lives inside — and two ways to write its header.

module / endmodule & Ports

Every design unit — however large — is delimited by exactly this pair of keywords.

ANATOMY OF A MODULE

```
module and_gate (  
    input wire a, // a port — a connection point to the outside world  
    input wire b,  
    output wire y  
);  
    assign y = a & b; // the body — what the module does  
endmodule
```

module <name> (<port list>); is a function signature for hardware — the name and ports are the interface; everything before **endmodule** is the implementation.

Comments, Whitespace & Identifiers

The small syntax rules that make everything else readable.



Comments

`//` runs to end of line. `/* ... */` spans multiple lines. Verilog is otherwise silent about documentation — a comment header naming each module's purpose is a house convention, not a language rule.



Whitespace

Free-format — spaces, tabs, and newlines are interchangeable and don't affect meaning. Indentation is purely for humans, same as in C



Case sensitivity

Verilog is case-sensitive: `and_gate` and `AND_GATE` are different identifiers. Keywords (`module`, `input`, `assign...`) are always lowercase.

Data Types

Four possible values, two fundamentally different kinds of signal.

Nets vs. Variables

The theory behind Lab 1's #1 rule: wire for assign, reg for always.

Nets — wire

A net represents a physical wire. It has no value of its own — it continuously reflects whatever drives it (a gate, a module port, an assign statement). Declared with wire (or the rarer wor/wand/tri).

```
wire y; assign y = a & b;
```

Variables — reg, integer, real, time

A variable holds a value between procedural assignments. reg is the workhorse; integer, real, and time exist for testbench convenience. Despite the name, reg does not always mean a hardware flip-flop.

```
reg y; always @(*) y = a & b;
```

Vectors & Buses

Bundling bits into a multi-bit signal — and picking individual bits back out.

DECLARING AND INDEXING A VECTOR

```
wire [7:0] data; // an 8-bit vector, bit 7 = MSB, bit 0 = LSB
```

```
data[0] // bit-select — a single bit
```

```
data[7:4] // part-select — the top nibble
```

```
wire [3:0] nib = data[3:0]; // assigning a slice to a narrower wire
```

Convention: [MSB:LSB] — this course always declares vectors most-significant-bit first, e.g. [7:0], never [0:7].

Arithmetic & Relational Operators

The same symbols you already know from software — with one hardware caveat.

Arithmetic

+ - * / % — addition, subtraction, multiplication, division, modulo. Division and modulo are simulation-friendly but expensive (or unsynthesizable) in real hardware — used sparingly outside testbenches.

```
sum = a + b; rem = a % b;
```

Relational

< > <= >= — compare magnitude, always return a single bit: 1 (true), 0 (false), or x if either operand contains an x.

```
wire gt = (a > b);
```

Logical & Bitwise Operators

Two families that look similar and behave completely differently.

Logical && || !

Treat their entire operand as one true/false value (any nonzero = true) and always return a single bit. Used for conditions — if, while.

```
if (a && !b) ...
```

Bitwise & | ^ ~ ~

Operate bit-by-bit across a vector, returning a vector the same width as the operands. This is what `and_gate`'s assign `y = a & b`; actually uses.

```
wire [3:0] y = a & b; // per-bit AND
```

The assign Statement

"Continuous" means exactly that — this line re-evaluates the instant its inputs change.

CONTINUOUS ASSIGNMENT

```
wire y;
```

```
assign y = a & b; // re-fires on every change to a or b
```



No clock, no trigger, no waiting

This is what makes assign the right tool for combinational logic — a real AND gate has no concept of "running once." It just always reflects its inputs, and assign models that directly.

Exercise 1 • Build an AND Gate

One module. Two inputs. One output. The simplest possible Verilog.

```
and_gate.v

// 2-input AND gate
module and_gate (
    input  wire a,
    input  wire b,
    output wire y
);

    assign y = a & b;

endmodule
```

NOTES

assign

is continuous — y updates the moment a or b changes.

&

is bitwise AND. For 1-bit signals, that's just AND.

wire

ports default to wire — you can omit it. Shown for clarity.

Exercise 1 • Write the Testbench

Drive every input combination. Watch the output. Record the waveform.

```
and_tb.v

module and_tb;
    reg  a, b;
    wire y;
    and_gate dut (.a(a), .b(b), .y(y));

    initial begin
        $dumpfile("and.vcd");
        $dumpvars(0, and_tb);
        a=0; b=0; #10;
        a=0; b=1; #10;
        a=1; b=0; #10;
        a=1; b=1; #10;
        $finish;
    end
endmodule
```

KEY POINTS

Testbench has no ports — it's the top of the simulation.

``reg`` for the inputs because they're driven inside a procedural block.

``#10`` = wait 10 time units before next change.

``$dumpvars`` records every signal under `and_tb` into the VCD.

Exercise 2 • Add OR, NOT, XOR

Same pattern as AND. Just three more modules. Three operators to remember.

```
or_gate.v

module or_gate (
    input  a,
    input  b,
    output y
);
    assign y = a | b;
endmodule
```

```
not_gate.v

module not_gate (
    input  a,
    output y
);
    assign y = ~a;
endmodule
```

```
xor_gate.v

module xor_gate (
    input  a,
    input  b,
    output y
);
    assign y = a ^ b;
endmodule
```

OPERATORS • & AND | OR ~ NOT ^ XOR

Common Errors • And the Fix

Four mistakes that will eat 30 minutes if you don't recognize them.

syntax error near `endmodule`

Missing semicolon at the end of an `assign` line. Verilog won't tell you the right line — look one above.

yourgate.v: undefined identifier `and\_gate`

You forgot to compile and_gate.v alongside the testbench. Pass BOTH files to iverilog.

VCD opens but is empty in GTKWave

You forgot `\$dumpfile` and `\$dumpvars` in the testbench. Without them, nothing is recorded.

Output stuck at X (red)

An input was never assigned. In Verilog, unassigned wires default to X (unknown), not 0.

Take These Home

Three short stretch problems. Solutions discussed at the start of Lab 2.

01

NAND and NOR from primitives

Implement a NAND gate using only the AND and NOT modules from this lab. Verify against its truth table.

02

Add an XNOR module

Add `xnor\_gate.v` to your set. Update top.v and the testbench to exercise all five gates together.

END OF LAB 01

Seven gates.
A whole CPU ahead.

Questions?



**Thanks
for
watching!**